

Milestone One: Functional Requirements

Project 2: Integration with Payment Provider & Ecommerce Payment Flows

Team 10: Felix Botero, Prisca Louis, Erik Gijlstra, Mariah Weldon

Purpose of the System:

The purpose of this project is to design and implement a simulated e-commerce payment system for Manhattan Associates. The system will allow customers to place orders and submit payment details, validate those details, authorize transactions via a mock payment provider, and store authorization results. Warehouse staff will then be able to finalize orders by validating settlement amounts against the authorized limit. The system will also allow the viewing of current & past orders in an order database.

Scope:

Main Goals:

- Customer checkout UI to collect order and payment details.
- Client-side validation for card expiration, card type detection, and masking of sensitive data.
- Integration with Manhattan Associates' mock payment authorization API.
- Handling of multiple authorization outcomes (success, insufficient funds, incorrect details, server error).
- Relational database storage of authorization results.
- Orders list with sorting and filtering capabilities.
- Warehouse settlement UI to validate and finalize orders.

Bonus/Optional Goals:

- Product catalog or shopping cart functionality.
- User login and authentication.

Definitions & Acronyms:

- **FR:** Functional Requirement

- **Order ID:** System-generated unique identifier for each order.
- **Authorization Token:** Static token returned by the mock API; stored in concatenated form with the Order ID.
- **Settlement:** Process of validating and finalizing an order against the authorized amount.
- **Mock API:** Simulated endpoint provided by Manhattan Associates to mimic payment authorization.

System Overview:

Major Components:

- **Customer Module:** Checkout process, payment validation, authorization handling.
- **Order Management Module:** Storage and display of transaction results with sorting/filtering.
- **Warehouse Module:** Settlement process to validate shipment amounts against authorizations.

Actors:

- **Customer:** Enters order and payment details.
- **Warehouse User:** Finalizes orders by entering final amounts.
- **System:** Validates input, handles API interactions, stores results, and enforces business rules.
- **Payment Provider (Mock API):** External service returning randomized authorization responses.

Functional Requirements:

Each requirement is listed with a description, rationale, and acceptance criteria.

FR-1: Order Entry

- **Description:** The system shall generate an Order ID and allow customers to enter personal, address, and payment details.
- **Rationale:** Captures orders and payment information to initiate the transaction process.

- **Acceptance Criteria:** A customer can input all required fields and submit an order for authorization.

FR-2: Client-Side Validation

- **Description:** The system shall validate payment details at the time of entry.
 - Detect card type (Visa, MasterCard, Amex).
 - Validate expiration date is in the future.
 - Mask card number (show only last 4 digits) and CVV.
- **Rationale:** Ensures correct and secure handling of payment data.
- **Acceptance Criteria:** Invalid or expired cards are rejected before submission; masked fields display correctly.

FR-3: Payment Authorization

- **Description:** The system shall call the mock API endpoint to authorize payments.
- **Rationale:** Simulates the real-world credit card authorization process.
- **Acceptance Criteria:** The system correctly submits data and displays the outcome.

FR-4: Handle Authorization Outcomes

- **Description:** The system shall handle and display the four API outcomes:
 - Success (60%)
 - Insufficient funds (17%)
 - Incorrect details (17%)
 - Server error (6%)
- **Rationale:** Provides realistic simulation of payment flows.
- **Acceptance Criteria:** Correct message is shown for each outcome, and the result is recorded in the database.

FR-5: Persist Authorization Data

- **Description:** The system shall store transaction details in a relational database.
- **Rationale:** Enables order management and settlement.
- **Acceptance Criteria:** Each authorization is stored with: Order ID, date/time, concatenated authorization token, amount, expiration, and status.

FR-6: Orders List

- **Description:** The system shall display all stored orders with sorting and filtering capabilities.
- **Rationale:** Improves order management and tracking for users.
- **Acceptance Criteria:** Users can sort orders by date, amount, or status, and filter by status.

FR-7: Settlement Entry

- **Description:** The system shall provide a UI for warehouse users to enter an Order ID and final shipment amount.
- **Rationale:** Allows settlement of orders against authorized amounts.
- **Acceptance Criteria:** Warehouse users can submit settlement amounts for any stored Order ID.

FR-8: Settlement Validation

- **Description:** The system shall compare the final shipment amount against the authorized amount.
- **Rationale:** Ensures settlements do not exceed authorization.
- **Acceptance Criteria:**
 - If final amount > authorized → display error, mark settlement as failed.
 - If final amount ≤ authorized → display success, mark as settled.

Data Flows:

1. **Customer places order** → enters details → system validates → sends to endpoint → receives response → displays result → stores record.
2. **Warehouse processes order** → enters Order ID + amount → system compares to stored authorization → updates status accordingly.
3. **Orders list view** → pulls stored data and applies sorting/filtering for user display.

Business Rules:

- Authorization tokens are static but stored uniquely by concatenating with Order ID.
- Expiration validation must occur within the system; API does not perform this check.

- Settlement cannot exceed authorization amount.
- Orders must always store a valid status (Authorized, Failed, Settled).

Assumptions & Constraints:

- No real credit card data will be used (mock only).
- Database will be relational (SQL-based).
- UI theming is optional; functional clarity is sufficient.
- Bonus features (e.g., cart, login, advanced filters) may be included if time permits.

Meeting Insights (Sept 5, 2025 – Zach, Manhattan Associates Developer):

- **Priority:** Back-end functionality first; UI clean but minimal.
- **Database:** SQL relational DB recommended; diagrams expected.
- **Masking:** Last 4 digits only; CVV masked; focus on UI masking.
- **Expiration:** Must be checked internally, not by API.
- **Tokens:** Static token from API; concatenate with Order ID.
- **Success Tiers:**
 - Minimum: functional flow + 3 UIs.
 - Elevated: cleaner checkout UI + solid order/warehouse UIs.
 - Bonus: login, cart, advanced filters, etc.
- **Documentation:** User manual not required, but optional for bonus points.
- **Branding:** UI may be themed (real or fake brand) but not required.
- **Pricing:** Static or random acceptable; full product/cart = bonus feature.

AI Use Disclaimer:

In the development of this Functional Requirements Document for Milestone 1, our team used ChatGPT (GPT-5) as a supportive tool. AI assistance was used for the following purposes:

- **Understanding project expectations:** We provided excerpts from the capstone guidelines and the Manhattan Associates project brief, and AI summarized them into clear deliverables.

- **Generating clarifying questions:** AI helped draft and explain potential questions to ask Manhattan Associates, then clarified technical concepts (e.g., admin panel vs. role-based interface, masking methods, authorization tokens).
- **Meeting transcript synthesis:** We supplied the transcript from our meeting with the Manhattan Associates developer, and AI summarized the key points into actionable project insights.
- **Drafting the requirements document:** AI assisted in creating a structured draft of the Functional Requirements Document, following the professor's guide and Manhattan Associates' example formatting.
- **Refining the document:** AI incorporated meeting insights, clarified business rules, and adjusted the outline to match professional standards.

All final content, structure, and decisions included in this document were reviewed and approved by our team.